



LectureAssist: An Agentic, Multimodal AI Assistant that Turns Lectures into Adaptive Study Material

Iftikhar Ahmed

iftikhar.ahmed@northsouth.edu

Salman Shafi

salman.shafi@northsouth.edu

Jarinah Tasnim

jarinah.tasnim@northsouth.edu

Dept. of Electrical and Computer Engineering, North South University • Senior Design Project (499) • Supervisor: Dr. Adnan Areefen

Problem & Goal

Recorded lectures pack speech, slides and board work into one video, and that content stays **locked and unsearchable**. Students re-watch hours of footage to find a single concept and have no fast way to self-test on what the lecturer actually emphasised.

- Lecture knowledge lives in **video, speech and board writing**, rarely as clean text.
- Generic quiz tools **hallucinate** off-topic, ungrounded questions.
- One-size-fits-all difficulty ignores **each student's weak spots**.

Goal. A system that *reads the board and hears the lecture*, and produces *grounded, validated, adaptive* study material entirely on a free Colab T4 at **zero API cost**.

System Architecture

A lecture video or document is extracted, indexed, and handed to an orchestrator that supervises specialist agents. Independent agents run concurrently; any agent crash is caught and logged so the pipeline **degrades gracefully** rather than failing outright.

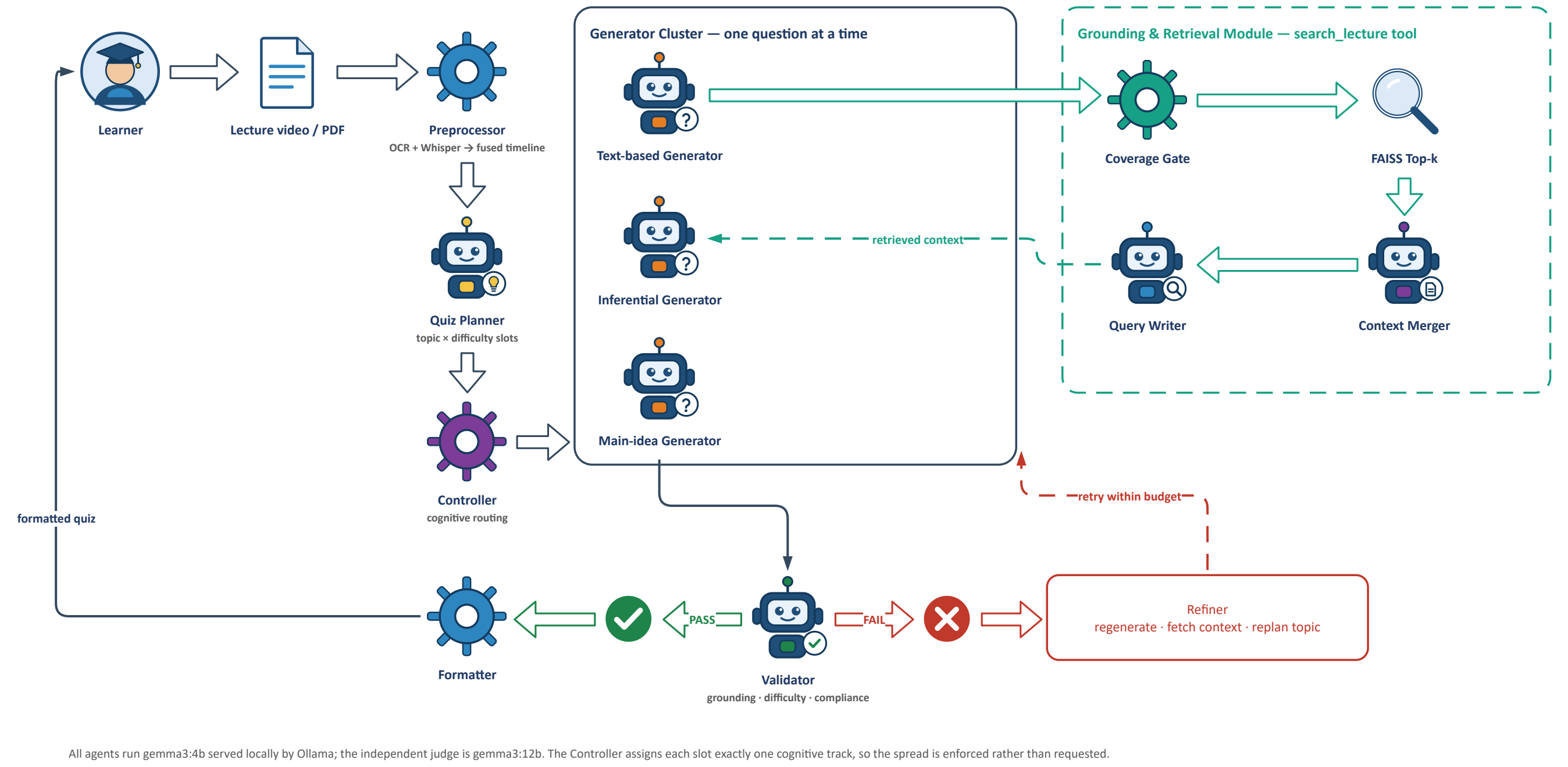


Figure 1. LectureAssist system architecture. Multimodal extraction feeds a grounded knowledge base; the orchestrator coordinates the specialist agents that produce every study artifact.

Stack

- **Vision / OCR.** EasyOCR, frames sampled ~ 5 s apart and classified (slide / blackboard / whiteboard / scene) with per-class preprocessing — CLAHE, sharpening, Otsu, chalk inversion.
- **Speech.** Faster-Whisper, GPU `float16` with CPU `int8` fallback, per-segment timestamps.
- **Retrieval.** Sentence-Transformers (`MiniLM`) embeddings in a `FAISS` cosine index; every artifact retrieves top- k chunks before generation.
- **Models.** Generator `gemma3:4b` (pinned), judge and answer-generation `gemma3:12b`, served locally by Ollama.
- **Serving.** Flask on Colab $\rightarrow$ ngrok $\rightarrow$ single-page browser UI.

The Agentic Loop

Questions are built **one at a time** and must *pass validation* before the next slot begins. On failure the validator selects a remediation — regenerate, fetch more context, or replan the topic. When the retry budget is exhausted the best attempt is kept and **marked as an override**; a failed question is never silently relabelled as a pass.

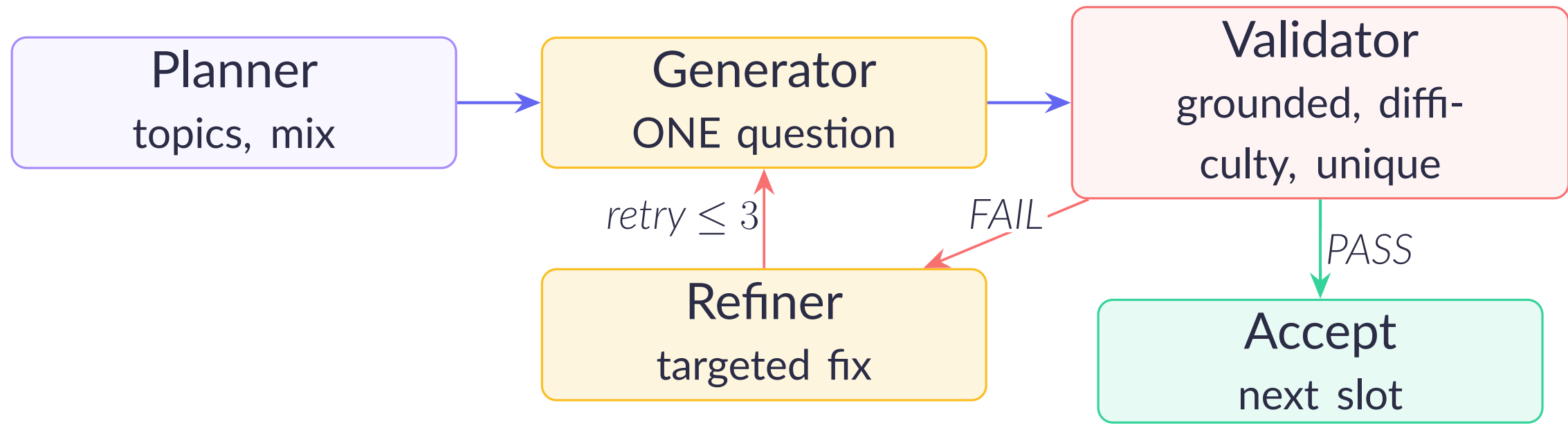


Figure 2. Plan $\rightarrow$ Generate $\rightarrow$ Validate $\rightarrow$ Retry, budget-capped at three attempts per slot.

How We Evaluated It

Generation and measurement are kept **strictly separate**: the Colab run only emits questions, and judging happens offline on the exported artifact.

- **Source.** *Limits* chapter of OpenStax *Calculus, Vol. 1* (≈ 112 k characters), ingested through the standard document pipeline and also supplied to the judge as ground truth.
- **Scale.** $n=200$ MCQ per pipeline, split across three target difficulties (easy 68, medium 66, hard 66).
- **Fairness.** Both pipelines read the document through an identical sliding window with a uniform batch size, so neither sees more or different content — any difference reflects **prompting strategy alone**.
- **Judge.** `gemma3:12b`, scoring questions written by the `gemma3:4b` generator.

Results by Difficulty

Diff.	Pipeline	n	Dup.↓	Ground.	Eff. acc.	Match
Easy	Non-agentic	68	0.353	0.562	0.647	0.971
	Agentic	68	0.324	0.642	0.676	0.971
Medium	Non-agentic	66	0.379	0.595	0.612	0.879
	Agentic	66	0.227	0.614	0.773	0.955
Hard	Non-agentic	66	0.288	0.513	0.712	0.970
	Agentic	66	0.182	0.616	0.818	0.985

Table 1. The pattern is consistent at every difficulty level: the agentic pipeline leads on duplicate rate, grounding and effective acceptance throughout.

Status & Next Steps

Configuration	Status
agentic, nonagentic	scored, $n=200$ each
cot, pot	implemented, not judged
merge_vr, merge_pg	built, <i>pending</i>
5 leave-one-agent-out configs	specified, <i>pending</i>
EQGBench / ReQUESTA protocols	harnesses built, <i>pending</i>

Table 2. No number is claimed for a run that has not completed. Next up are the leave-one-out ablations, which test the duplication hypothesis.

Headline Results

Metric	Scale	Non-agentic	Agentic
<i>Judged quality — all four saturate</i>			
Overall quality	1–5	4.86	4.93
Correctness	1–5	4.98	5.00
Clarity	1–5	4.92	4.97
Difficulty match	1–5	4.36	4.39
<i>Automated — these actually discriminate</i>			
Duplicate rate ↓	0–1	0.385	0.280
Semantic grounding	0–1	0.557	0.624
Effective acceptance	0–1	0.612	0.720
Independent answer match	0–1	0.940	0.970
Inter-question similarity ↓	0–1	0.932	0.894
Acceptance rate	0–1	0.995	1.000
Judge-verified correctness	0–1	0.995	0.995

Table 3. Native metrics, $n=200$ per pipeline. Green marks the better value; arrows mark rows where lower is better.

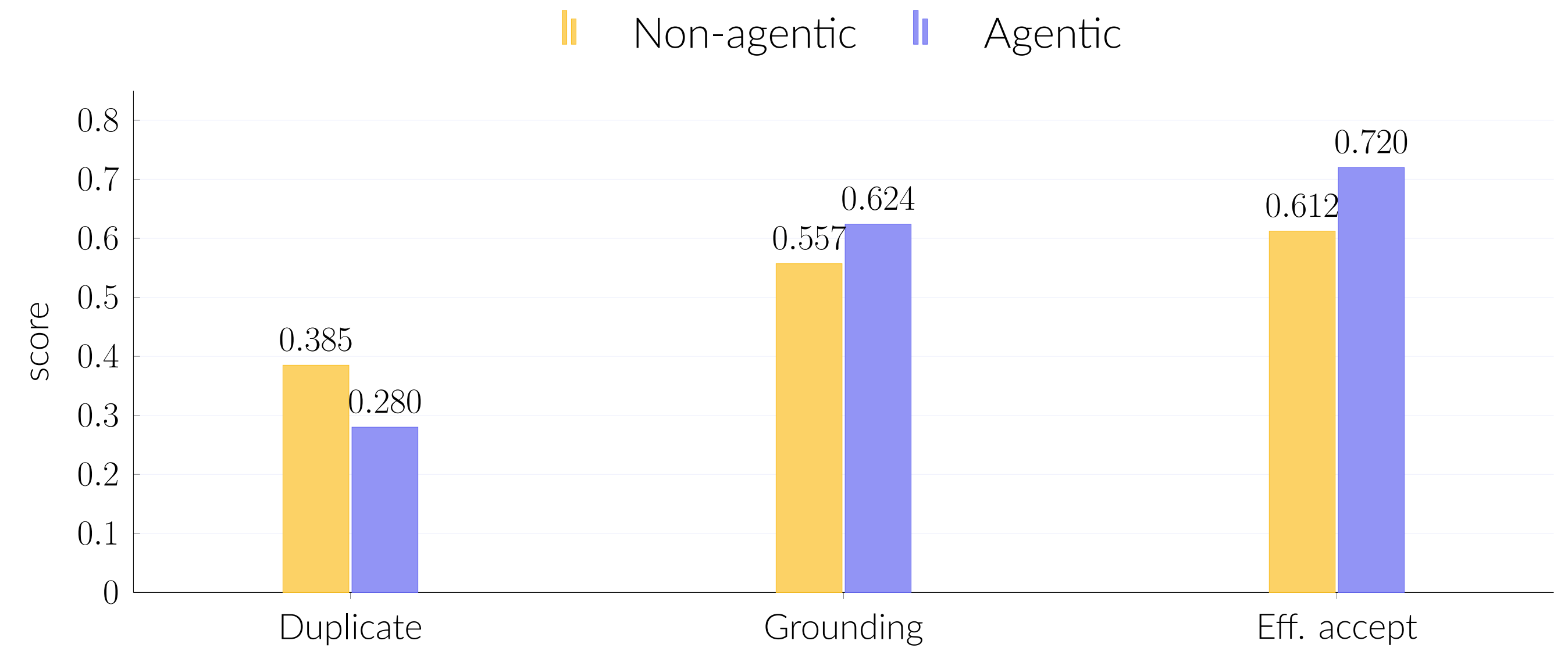


Figure 3. The three discriminating metrics. On duplicate rate *lower* is better, so the agentic pipeline leads on all three.

What the Numbers Actually Say

On this instrument the agentic pipeline **outperforms** the single-pass baseline. It matches or beats the baseline on every judged dimension and leads on all three metrics that discriminate.

Why the quality scores are uninformative. All four judged dimensions sit between 4.36 and 5.00 on a five-point scale, and correctness reaches the ceiling exactly. *A metric that cannot go up cannot discriminate* — the same saturation EQGBench documents for four of its own dimensions. The agentic advantage therefore rests entirely on the automated metrics, not on the judged ones.

Mechanism — stated as a hypothesis, not a result. Validation and retry reject a draft that repeats an accepted question or is weakly grounded, and the retry is fed the failure reason, so the loop suppresses duplication and pulls generation back toward the source material. This is consistent with the data but is **not established by it**; the —refiner and —planner ablations are designed to test which agent is responsible.

Conclusion

LectureAssist delivers a fully local, \$0-API multimodal pipeline that turns a lecture into grounded, validated, adaptive study material. On the one completed run the agentic loop beats single-pass prompting on every metric that discriminates — duplication,